2 академических часа

Введение в контейнеризацию. Архитектура Docker

Лекция №1 · Docker и контейнеризация приложений

Цель лекции

- Понять принципы контейнеризации и устройство Docker, чтобы осмысленно работать с образами и контейнерами.
- Разобрать проблему несовместимости окружений.
- Сравнить виртуальные машины и контейнеры.
- Изучить Docker Client, Docker Daemon, Images, Containers и Registry.
- Подготовиться к запуску первого контейнера.

Что необходимо знать до занятия

- ОС Linux: процессы, каталоги, терминал и права доступа.
- Основы HTTP: браузер, веб-сервер, порт, ответ.
- Назначение Node.js, npm и базы данных.
- Разница между хостовой и гостевой системой.

Учебные вопросы

1	**Шаг 1**	1. Проблемы развёртывания ПО.
2	**Шаг 2**	2. Понятие и принципы контейнеризации.
3	**Шаг 3**	3. Контейнеры и виртуальные машины.
4	**Шаг 4**	4. Архитектура Docker.
5	**Шаг 5**	5. Образы, контейнеры и реестры.
6	**Шаг 6**	6. Docker в разработке и эксплуатации.

Проблемы развёртывания программ

Почему приложение работает у разработчика, но не на сервере?

Ситуация: REST API работает только локально

- Разработчик запускает Express API на своём компьютере, но на другом ПК приложение не стартует.
- На сервере другая версия Node.js.
- Отсутствуют установленные npm-пакеты.
- Не заданы переменные окружения.
- PostgreSQL отсутствует или недоступен.
- Системные библиотеки и настройки отличаются.

Традиционное развёртывание REST API

1	**Шаг 1**	Установить и настроить ОС.
2	**Шаг 2**	Установить совместимые Node.js и npm.
3	**Шаг 3**	Перенести исходный код приложения.
4	**Шаг 4**	Установить npm-зависимости.
5	**Шаг 5**	Установить и настроить PostgreSQL.
6	**Шаг 6**	Передать конфигурацию, проверить сеть и запустить API.

Почему ручное развёртывание сложно поддерживать?

- В разных средах появляются несовпадающие версии зависимостей.
- Повторение многочисленных операций повышает риск ошибки.
- Обновления системы могут изменить поведение приложения.
- Для тестирования требуется воспроизводимое окружение.

Главная идея

Цель: описать среду выполнения так, чтобы запуск можно было повторять.

Что предлагает Docker?

- Приложение и необходимые зависимости упаковываются в образ, из которого затем создаются контейнеры.
- Один образ можно использовать на разных совместимых хостах.
- Сборку и запуск можно автоматизировать.
- Конфигурацию и постоянные данные следует учитывать отдельно.
- Docker повышает воспроизводимость, но не гарантирует работу на любой ОС и архитектуре.

От исходного кода к контейнеру

Как приложение и его зависимости превращаются в Docker-образ и запускаются в контейнере

1 Исходный код приложения

Разработчик пишет код и описывает зависимости.

```
my-app/
  src/
    server.js
    package.json
  .dockerignore
  Dockerfile
```

package.json (фрагмент)

```
{
  "name": "my-app",
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

2 Dockerfile

Описывает, как собрать образ: базовый образ, копирование файлов, установка зависимостей и команда запуска.

Dockerfile

```
FROM node:22-alpine

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000
CMD ["node", "src/server.js"]
```

3 Сборка образа

Docker читает Dockerfile, выполняет инструкции и создаёт слоёный образ.

Команда сборки

```
$ docker build -t my-app:1.0 .
```

Процесс сборки

- Загрузка базового образа
- Установка зависимостей
- Копирование исходного кода
- Создание слоёв образа
- Готовый образ my-app:1.0

4 Docker-образ

Результат сборки — неизменяемый образ, состоящий из слоёв.



my-app:1.0
Docker Image

5 Запуск контейнера

Из образа создаётся контейнер, в котором запускается приложение.



Команда запуска

```
$ docker run -d \
    --name my-app \
    -p 3000:3000 \
    my-app:1.0
```

Контейнер запущен
Приложение доступно на http://localhost:3000

Исходный код
Файлы приложения и зависимости

→

Dockerfile
Инструкции для сборки

→

Docker-образ
Неизменяемый шаблон

→

Контейнер
Запущенное приложение в изолированной среде

Контейнеризация и принципы работы

Контейнер — изолированная среда выполнения процессов

Что такое контейнер?

- Контейнер — изолированное окружение для выполнения одного или нескольких процессов приложения.
- Контейнер создаётся из образа.
- Содержит нужную файловую систему приложения и зависимости.
- Может иметь собственное представление о процессах и сети.
- Linux-контейнер использует ядро хостовой Linux-системы.

Что обычно входит в контейнер?

- Исполняемый файл приложения или интерпретатор.
- Библиотеки и вспомогательные файлы.
- Конфигурационные файлы и рабочие каталоги.
- Основной процесс: nginx, node либо postgres.

Главная идея

Контейнер обычно не содержит отдельного ядра операционной системы.

Контейнер — не маленькая виртуальная машина

- Процессы контейнера выполняются ядром хоста.
- Изоляция ресурсов не равна аппаратной виртуализации.
- Контейнер может содержать несколько процессов.
- После завершения основного процесса контейнер обычно переходит в остановленное состояние.

Linux namespaces: что изолируется?

Namespace	Что изолирует
PID	Идентификаторы и видимость процессов
NET	Сетевые интерфейсы, маршруты, порты
MNT	Представление точек монтирования
UTS	Hostname и доменное имя
IPC	Механизмы межпроцессного взаимодействия
USER	Идентификаторы пользователей и групп

Cgroups: учёт и ограничение ресурсов

- Control Groups (cgroups) — механизм ядра Linux для группировки процессов и управления использованием ресурсов.
- Учёт процессорного времени и памяти.
- Возможность задавать ограничения ресурсов.
- Защита соседних сервисов от неконтролируемого потребления ресурсов.
- Сами по себе cgroups не создают сетевой или файловой изоляции.

Namespaces и cgroups — разные задачи

Namespaces

- Определяют, что видит процесс.
- Изоляция PID, NET, MNT и других пространств.
- Пример: собственный hostname контейнера.

Cgroups

- Учитывают и ограничивают ресурсы.
- CPU, память и другие ресурсы.
- Пример: лимит памяти контейнера.

Ядро Linux, namespaces и cgroups

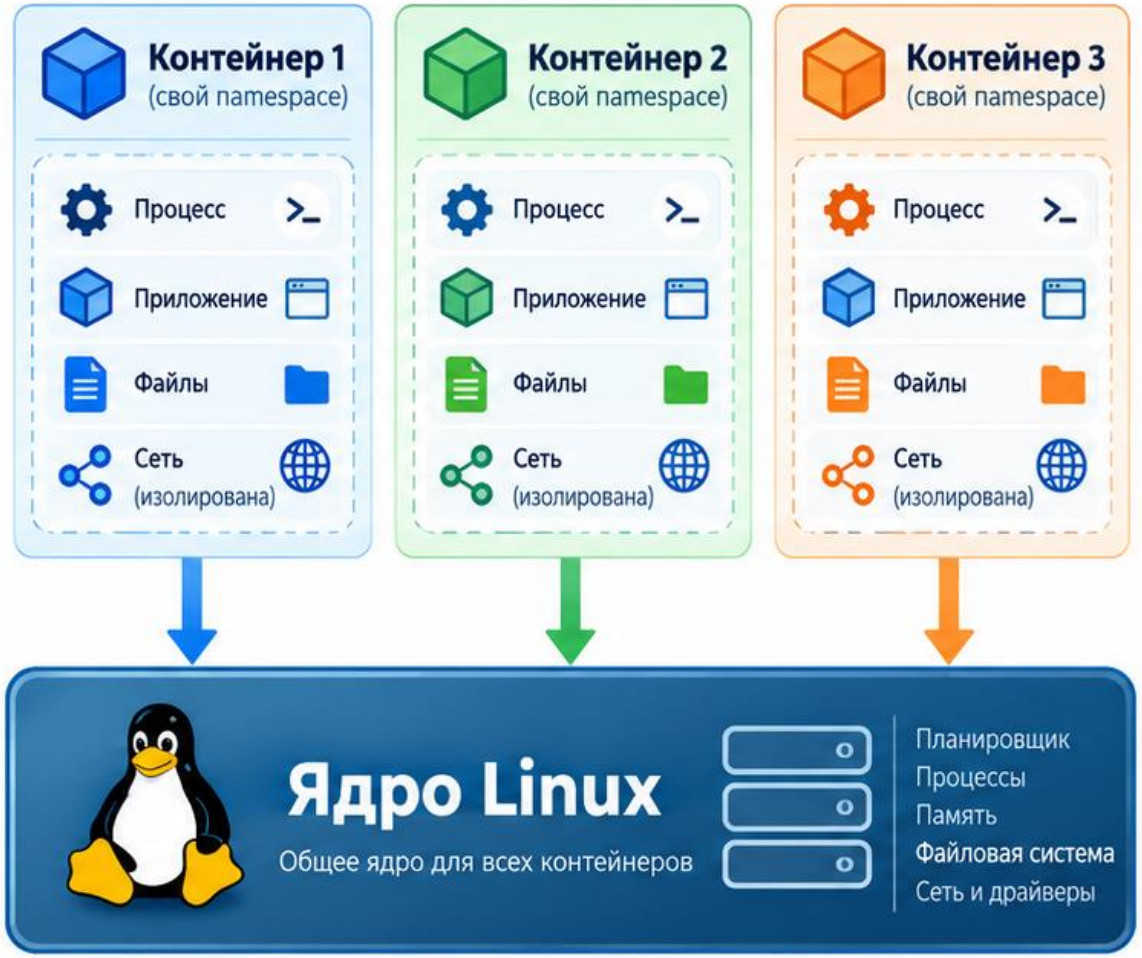
Как Linux-контейнеры обеспечивают изоляцию и контроль ресурсов

Namespaces = изоляция

Предоставляют контейнеру своё собственное представление о системных ресурсах.

- **PID** — изоляция процессов
- **NET** — изоляция сети
- **MNT** — изоляция точек монтирования
- **UTS** — имя хоста (hostname)
- **IPC** — межпроцессное взаимодействие
- **USER** — пользователи и группы

Каждый контейнер видит своё пространство ресурсов.



cgroups = контроль ресурсов

Позволяют ограничивать, учитывать и распределять использование ресурсов между контейнерами.

- **CPU** — ограничение процессора (50%)
- **RAM** — ограничение памяти (2 GB)
- **I/O** — контроль дисковых операций (100 MB/s)
- **PIDs** — лимит процессов (256)

Позволяют учитывать и ограничивать использование ресурсов.

1. **Контейнер =** изолированные процессы
2. **Namespaces** разделяют окружение
3. **cgroups** ограничивают ресурсы

Границы изоляции и безопасность

- Контейнеры разделяют ядро Linux и зависят от его безопасности.
- Привилегированные контейнеры повышают риски.
- Доступ к Docker Daemon предоставляет широкие полномочия на хосте.
- Позднее изучим непривилегированные процессы и ограничения ресурсов.

Главная идея

Контейнеризация не является полной заменой механизмов безопасности.

Контейнеры и виртуальные машины

Сравнение двух моделей изоляции

Как устроена виртуальная машина?

- Гипервизор предоставляет виртуальное оборудование, на котором запускается полноценная гостевая ОС со своим ядром.
- Примеры гипервизоров: VirtualBox, VMware, KVM.
- Внутри ВМ выполняются приложения и системные службы.
- Каждая ВМ использует ресурсы на собственную гостевую ОС.

Как устроена контейнерная среда?

- В Linux контейнеры выполняются как изолированные процессы, использующие ядро хостовой ОС.
- Не требуется загружать отдельное ядро для каждого контейнера.
- Контейнеры создают из заранее подготовленных образов.
- Linux-контейнеры на Windows обычно работают через Linux-окружение или ВМ.

Виртуальная машина и контейнер

Виртуальная машина

- Гостевая ОС со своим ядром.
- Аппаратная виртуализация.
- Обычно заметные расходы на гостевую ОС.
- Можно запускать иную гостевую ОС.

Контейнер

- Общее ядро Linux с хостом.
- Изоляция процессов и ресурсов.
- Обычно быстрее запуск и меньше накладные расходы.
- Среда зависит от платформы и архитектуры.

Архитектура ВМ и Docker-контейнеров

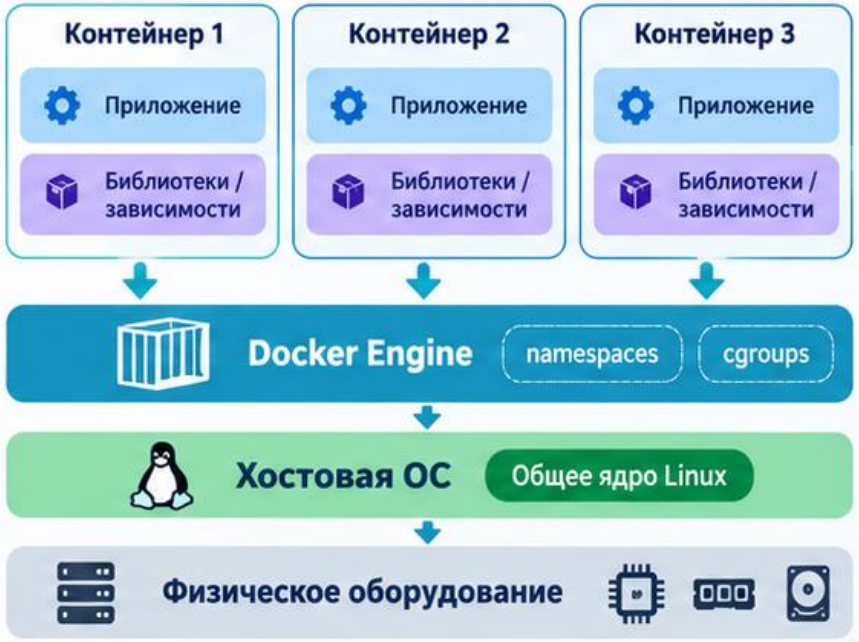
Сравнение уровней изоляции, компонентов и использования ресурсов

Виртуальные машины (VM)



- Каждая ВМ содержит собственную гостевую ОС
- Более высокая изоляция
- Больше потребление RAM и CPU
- Запуск обычно медленнее

Docker-контейнеры



- Контейнеры используют общее ядро хоста
- Лёгкая изоляция процессов
- Меньше потребление ресурсов
- Быстрый запуск

Критерий	Виртуальные машины (VM)	Docker-контейнеры
Изоляция	сильная	на уровне процессов
ОС внутри	полная гостевая ОС	нет отдельной гостевой ОС
Запуск	медленнее	быстрее
Ресурсы	больше	меньше
Типичное применение	разные ОС, полная виртуализация	развёртывание приложений

VM виртуализируют целую систему, а Docker контейнеризирует приложение и его зависимости.

Учебная среда курса: Windows → Debian → Docker

- Физический ПК: Windows 11.
- VirtualBox запускает виртуальную машину Debian 13.
- В Debian установлен Docker Engine.
- В Docker работают Nginx, Node.js и PostgreSQL.

Главная идея

Контейнер не заменяет Debian-ВМ: контейнеры запущены внутри неё.

Важно: localhost в разных системах

- localhost внутри Debian указывает на Debian.
- localhost на Windows указывает на Windows.
- Порт, опубликованный Docker в Debian, доступен в Debian через localhost.
- Для доступа из Windows требуется настройка сети ВМ или проброс портов VirtualBox.

Архитектура Docker

Клиент, демон, Engine и API

Docker Client

- Docker Client — интерфейс, через который пользователь отправляет команды Docker Daemon.
- Обычно применяется команда docker в терминале.
- CLI обращается к Docker API.
- Команда запускается в терминале, а контейнером управляет серверная часть Docker.

Первые команды клиента

```bash
docker --version
sudo docker version
sudo docker info
```

Что важно

- docker --version показывает версию установленной клиентской программы.
- docker version показывает сведения о клиенте и сервере; сервер должен быть доступен.
- docker info отображает параметры Docker Engine и его окружения.

Docker Daemon (dockerd)

- Серверный процесс Docker, который обрабатывает запросы клиента и управляет объектами Docker.
- Создание и запуск контейнеров.
- Загрузка и хранение образов.
- Работа с сетями и томами.
- В Linux обычно запускается как системная служба.

Проверка службы Docker в Debian

```bash
sudo systemctl status docker
```

Что важно

- systemctl обращается к диспетчеру служб systemd.
- status показывает текущее состояние службы.
- active (running) означает, что служба запущена; для выхода из просмотра нажмите q.

Docker Engine и компоненты выполнения

- Docker Engine предоставляет клиент-серверный механизм управления контейнерами.
- Docker Daemon принимает запросы через API.
- containerd участвует в управлении жизненным циклом контейнеров.
- OCI runtime (например, runc) обеспечивает непосредственный запуск контейнерных процессов.

Docker Client → Docker Daemon → Container

Как команда пользователя превращается в работающий контейнер

Docker Client

CLI / команда docker




```
docker run nginx
docker ps
docker images
```

Принимает команды пользователя.

Docker Daemon

dockerd



- Получает запрос от клиента
- Управляет образами
- Создаёт и запускает контейнеры
- Работает через Docker API

Фоновая служба на хост-системе, которая управляет Docker.

Container

Запущенный контейнер



- Приложение
- Библиотеки
- Изолированная среда

Выполняет задачу, например Nginx.

1 Пользователь вводит команду



2 Client отправляет запрос



3 Daemon обрабатывает запрос



4 Создаётся контейнер



5 Приложение работает



Что важно помнить

- **Client** — это интерфейс команд.
- **Daemon** — это служба Docker.
- **Container** — это запущенный экземпляр образа.

Client передаёт команду, Daemon выполняет её, Container запускает приложение.

Как выполняется docker run hello-world?

1	**Шаг 1**	Клиент отправляет запрос Docker Daemon.
2	**Шаг 2**	Daemon проверяет, есть ли нужный образ локально.
3	**Шаг 3**	При отсутствии образ загружается из реестра.
4	**Шаг 4**	Docker создаёт контейнер на основе образа.
5	**Шаг 5**	Контейнер запускает программу и выводит сообщение.
6	**Шаг 6**	Основной процесс завершается, контейнер останавливается.

Демонстрация: первый контейнер

```bash
sudo docker run hello-world
```

Что важно

- sudo выполняет команду с правами администратора Debian.
- docker run создаёт и запускает контейнер из выбранного образа.
- hello-world — имя образа; ожидается строка Hello from Docker!.
- После выполнения программы контейнер будет остановлен.

Docker Images, Containers и Registry

Образ — шаблон, контейнер — экземпляр

Docker Image

- Образ Docker — шаблон файловой системы и конфигурации, из которого создаются контейнеры.
- Формируется из слоёв; слои могут переиспользоваться.
- Включает файлы и зависимости приложения.
- Образ используют для создания одного или многих контейнеров.
- Для обновления версии приложения обычно создают новый образ.

Теги образов

- Форма имени: repository:tag.
- nginx:stable — канал стабильного образа Nginx.
- node:22 — образ Node.js с тегом 22.
- postgres:17 — PostgreSQL с тегом 17.
- Тег может со временем указывать на другую сборку; digest фиксирует конкретное содержимое.

Docker Registry и Docker Hub

- Registry — сервис хранения и выдачи контейнерных образов.
- Docker Hub — один из публичных реестров.
- docker pull загружает образ.
- docker push публикует образ при наличии доступа.
- Для публичных образов в лабораторной используем hello-world и nginx.

Образы и контейнеры: разные списки

BASH

```
sudo docker images
sudo docker ps
sudo docker ps -a
```

Что важно

- docker images показывает локальные образы.
- docker ps показывает работающие контейнеры.
- docker ps -a также показывает остановленные контейнеры.
- Наличие образа не означает наличие работающего контейнера.

Образ и контейнер

Образ (Image)

- Неизменяемый шаблон с файлами.
- Хранится локально или в реестре.
- Не является запущенным процессом.
- Один образ → несколько контейнеров.

Контейнер (Container)

- Конкретный экземпляр образа.
- Имеет имя, идентификатор и состояние.
- При запуске выполняет процессы.
- Можно остановить и удалить отдельно от образа.

Docker-образ (один образ)

Шаблон, который содержит файловую систему, приложение, зависимости и настройки.



- Файловая система Linux
- Приложение (например, Nginx)
- Зависимости (библиотеки)
- Настройки по умолчанию

Создание контейнера из образа

Создание контейнера из образа

Создание контейнера из образа



Контейнер 1
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)



Контейнер 2
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)



Контейнер 3
nginx:latest
▶ Запущен

- Собственная изолированная среда
- Свой процесс Nginx
- Свои сетевые настройки
- Свои изменяемые данные (внутри контейнера)

Жизненный цикл контейнера

- Created — контейнер создан, но не запущен.
- Running — выполняется основной процесс.
- Exited — основной процесс завершился или контейнер остановлен.
- После stop контейнер можно start или rm.
- Удаление контейнера не удаляет его образ автоматически.

Основные команды жизненного цикла

BASH

```
sudo docker ps -a
sudo docker stop web1
sudo docker start web1
sudo docker rm web1
```

Что важно

- ps -a показывает состояние контейнера.
- stop останавливает работающий контейнер.
- start повторно запускает остановленный контейнер.
- rm удаляет остановленный контейнер; выполняйте только для созданных вами объектов.

Веб-сервер в контейнере

BASH

```
sudo docker run -d --name web1 -p 8080:80 nginx:stabl
```

Что важно

- -d — запуск в фоновом режиме.
- --name web1 — имя контейнера.
- -p 8080:80 — порт Debian 8080 перенаправляется на порт контейнера 80.
- В Debian откройте http://localhost:8080; при успехе появится страница Nginx.

Как понимать публикацию порта?

- Параметр -p HOST_PORT:CONTAINER_PORT связывает порт хостовой системы с портом приложения в контейнере.
- В примере Nginx слушает порт 80 внутри контейнера.
- Клиент обращается к порту 8080 на Debian.
- Если порт 8080 уже занят, Docker не сможет опубликовать его.
- Без дополнительных ограничений опубликованный порт может быть доступен через сетевые интерфейсы хоста.

Применение Docker

От первого веб-сервера до итогового проекта

Docker в ежедневной работе программиста

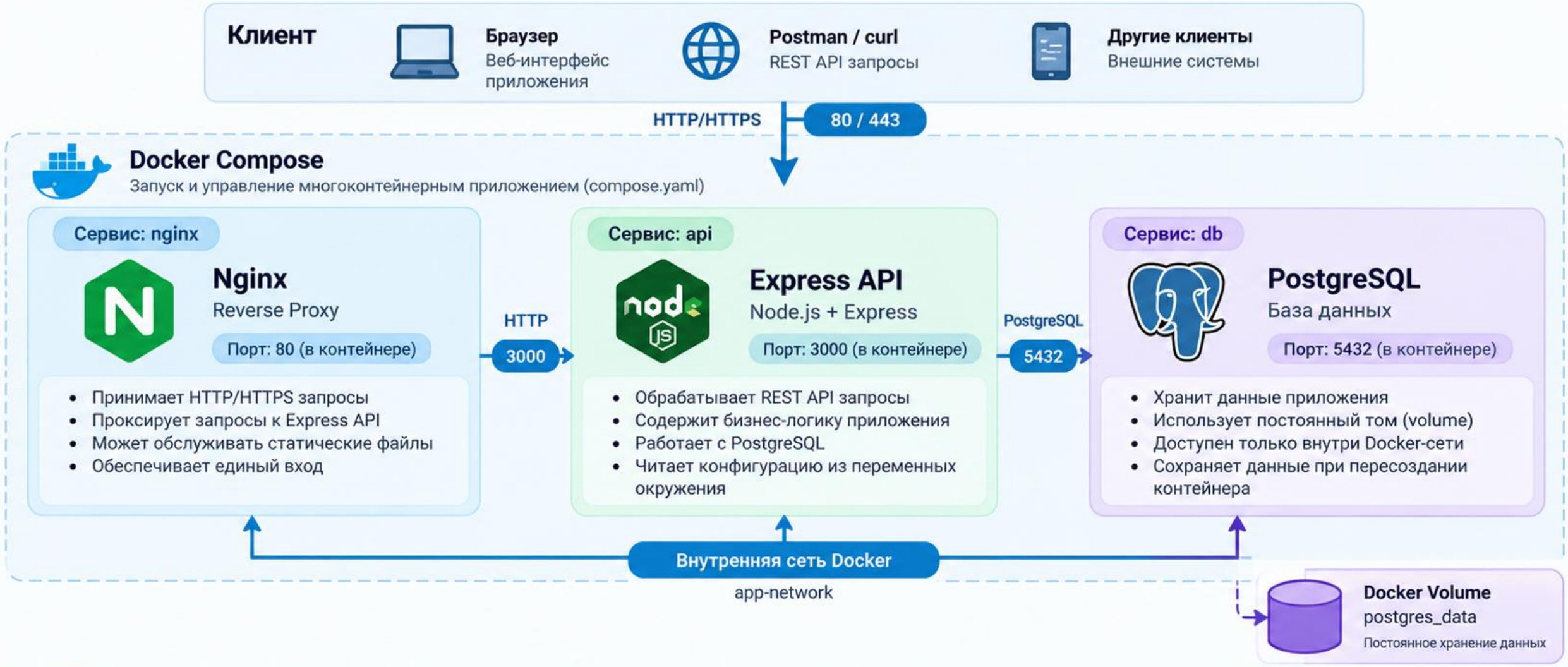
- Локальная разработка без установки каждой службы на хост.
- Повторяемые окружения для тестов.
- Сборка и доставка версионированных образов.
- Развёртывание сервисов на Linux-сервере.

Docker и архитектура приложения

- В одном контейнере может работать Express REST API.
- Во втором — PostgreSQL с отдельным постоянным томом.
- В третьем — Nginx как обратный прокси.
- Связь сервисов и совместный запуск настраиваются средствами Docker Compose.

Итоговый проект: веб-приложение на Docker

Архитектура и взаимодействие компонентов: Nginx → Express API → PostgreSQL



Ключевые преимущества итогового проекта

Единая команда запуска
docker compose up

Изолированные сервисы
Каждый компонент в отдельном контейнере

Сохраняемые данные
PostgreSQL в Docker Volume

Готово к развёртыванию
Можно запустить на Linux-сервере

Что предстоит в лабораторной №1?

- Проверить работоспособность установленного Docker Engine.
- Запустить и исследовать hello-world.
- Запустить Nginx с публикацией порта.
- Посмотреть логи и конфигурацию контейнера.
- Остановить, перезапустить и удалить контейнеры.
- Создать два независимых контейнера из одного образа.

Итоги лекции

1. Контейнеризация повышает воспроизводимость среды запуска приложений.
2. Linux-контейнеры используют общее ядро хоста; namespaces изолируют представления, cgroups управляют ресурсами.
3. Docker использует архитектуру Client → Daemon → контейнеры.
4. Image — шаблон; Container — экземпляр с собственным жизненным циклом.
5. Registry хранит образы; опубликованные порты обеспечивают доступ к сервисам.

Контрольные вопросы

1. Почему REST API может не запускаться на другом компьютере?
2. Чем namespaces отличаются от cgroups?
3. Почему контейнер не равен виртуальной машине?
4. Как взаимодействуют Docker Client и Docker Daemon?
5. Что делает команда docker run hello-world?
6. Чем отличаются docker images и docker ps -a?
7. Почему удаление контейнера не удаляет его образ?
8. Что означает параметр -p 8080:80?